

使用 Beam 上的 Privacy 計算私密統計資料

來源：<https://codelabs.developers.google.com/codelabs/privacy-on-beam?hl=zh-tw>

步驟數：10

在本程式碼研究室中，您將瞭解如何透過「Privacy on Beam」功能，產生餐廳造訪的私人統計資料，探索並應用差異化隱私架構的功能。

目錄

1. [簡介](#)
2. [差異化隱私權總覽](#)
3. [下載 Privacy on Beam](#)
4. [計算每小時的造訪次數](#)
5. [使用公開分區](#)
6. [計算平均入住天數](#)
7. [計算每小時收益](#)
8. [計算多項統計資料](#)
9. [\(選用\) 調整差異隱私權參數](#)
10. [摘要](#)

1. 簡介

您可能會認為匯總統計資料不會洩漏任何資訊，包括統計資料所依據的個人資料。不過，攻擊者可以透過匯總統計資料，以多種方式取得資料集中個人的私密資訊。

為保護個人隱私，您將瞭解如何使用 Privacy on Beam 的差異化隱私匯總功能，製作私人統計資料。Beam 上的 Privacy 是一項差異化隱私架構，可與 [Apache Beam](#) 搭配使用。

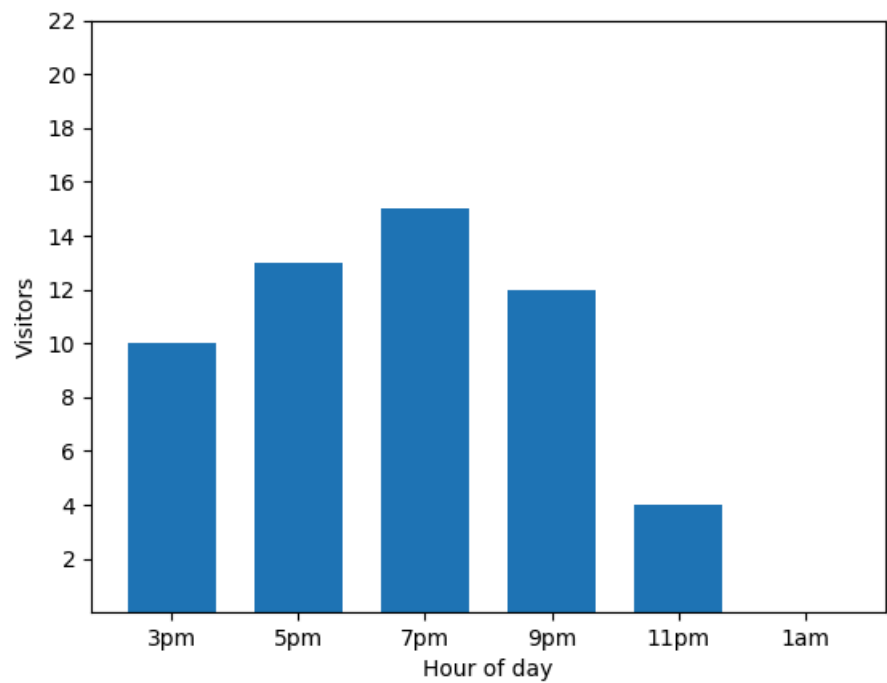
「私人」是什麼意思？

在本程式碼研究室中，我們使用「私密」一詞時，是指輸出內容的產生方式不會洩漏資料中個人的任何私密資訊。我們可透過差異化隱私 (一種強大的去識別化隱私概念) 達成此目標。去識別化是指匯總多位使用者的資料，以保護使用者隱私。所有去識別化方法都會使用匯總，但並非所有匯總方法都能達到去識別化效果。另一方面，差異化隱私可提供有關資訊外洩和隱私權的可測量保證。

2. 差異化隱私權總覽

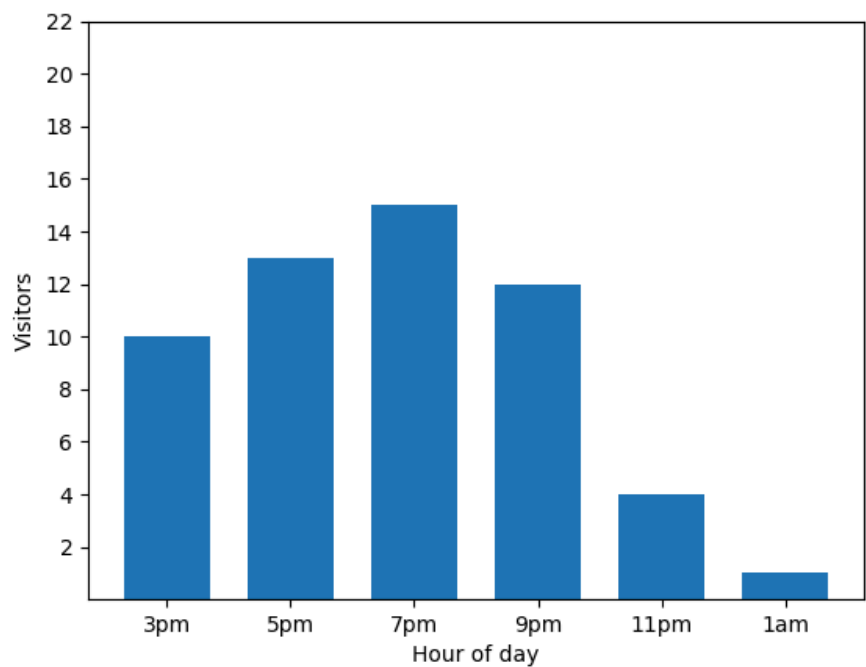
為了方便瞭解差異化隱私，我們來看一個簡單的例子。

這張長條圖顯示某個晚上小型餐廳的繁忙程度。許多顧客會在晚上 7 點光臨，而餐廳在凌晨 1 點完全沒有顧客：

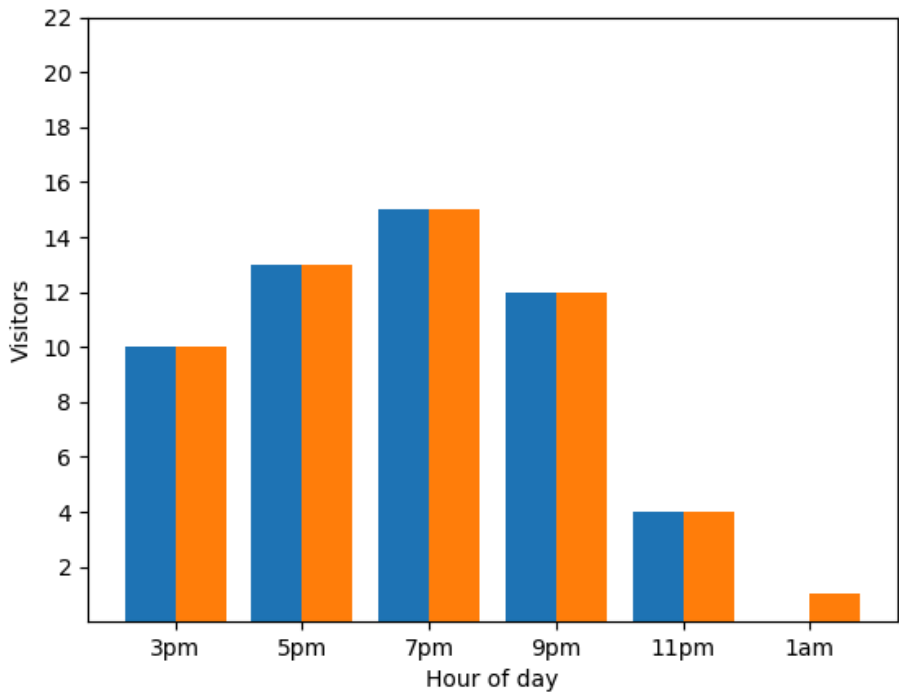


這項做法看起來很實用！

但有條件限制。**有新訪客抵達**時，長條圖會立即顯示這項資訊。查看圖表：很明顯有新訪客，而且這位訪客大約在凌晨 1 點抵達：



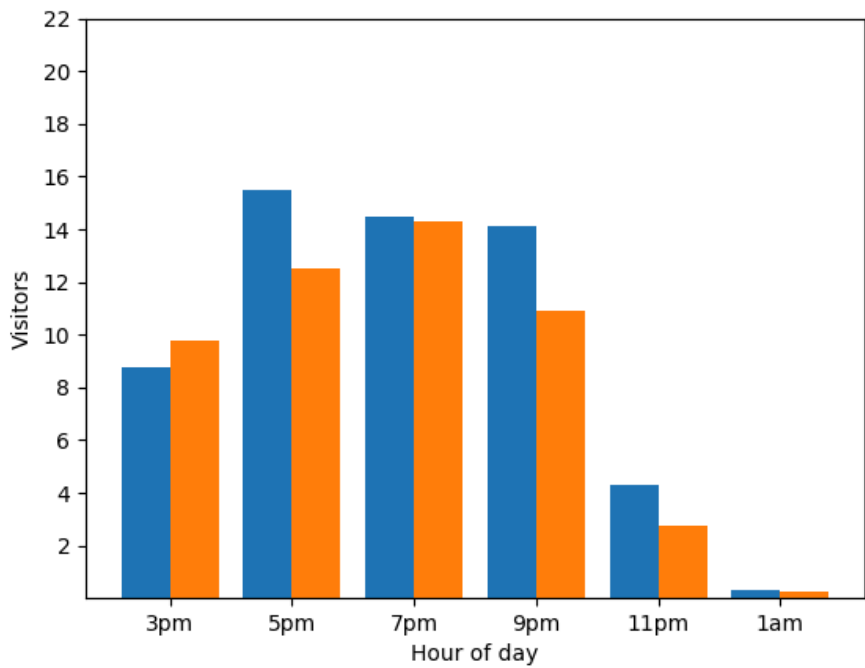
從隱私權的角度來看，這並非理想做法。真正匿名化的統計資料不應揭露個別貢獻。將這兩張圖表並列，就能更清楚看出差異：橘色長條圖多了一位在凌晨 1 點左右抵達的房客：



這同樣不是理想的結果。我們該怎麼做？

我們會加入隨機雜訊，讓長條圖的準確度稍微降低！

請參閱下方的兩張長條圖。雖然不完全準確，但仍有參考價值，且不會揭露個人貢獻。太棒了！



差異化隱私會在資料中加入適量的隨機雜訊，遮蓋個別貢獻內容。

我們的分析有些過於簡化。正確導入差異化隱私需要更多時間，且實作上會出現許多意想不到的細微差異。與密碼學類似，自行實作差異化隱私可能不是好主意。您可以改用 Beam 的隱私權功能，不必自行實作解決方案。請勿自行實作差異化隱私！

在本程式碼實驗室中，我們將說明如何使用 Privacy on Beam 執行差異化隱私分析。

步驟 3 · 約 0 分鐘

3. 下載 Privacy on Beam

您不需要下載 Privacy on Beam，即可完成本程式碼研究室，因為所有相關程式碼和圖表都可以在這份文件中找到。不過，如果您想下載程式碼來執行或使用 Privacy on Beam，請按照下列步驟操作。

請注意，本程式碼研究室適用於程式庫 1.1.0 版。

首先，請下載 Beam 上的 Privacy：

<https://github.com/google/differential-privacy/archive/refs/tags/v1.1.0.tar.gz>

或者，您也可以複製 GitHub 存放區：

```
git clone --branch v1.1.0 https://github.com/google/differential-privacy.git
```

Beam 的隱私權位於頂層 `privacy-on-beam/` 目錄。

本程式碼研究室的程式碼和資料集位於 `privacy-on-beam/codelab/` 目錄中。

此外，您也必須在電腦上安裝 Bazel。前往 [Bazel 網站](#)，查看適用於您作業系統的安裝說明。

4. 計算每小時的造訪次數

假設您是餐廳老闆，想要分享餐廳的統計資料，例如揭露熱門時段。幸好您瞭解差異化隱私和匿名化，因此希望以不會洩漏任何訪客資訊的方式進行這項作業。

本範例的程式碼位於 `codelab/count.go` 中。

首先，請載入包含特定星期一餐廳訪客人數的模擬資料集。就本程式碼實驗室而言，這段程式碼並不有趣，但您可以在 `codelab/main.go`、`codelab/utils.go` 和 `codelab/visit.go` 中查看相關程式碼。

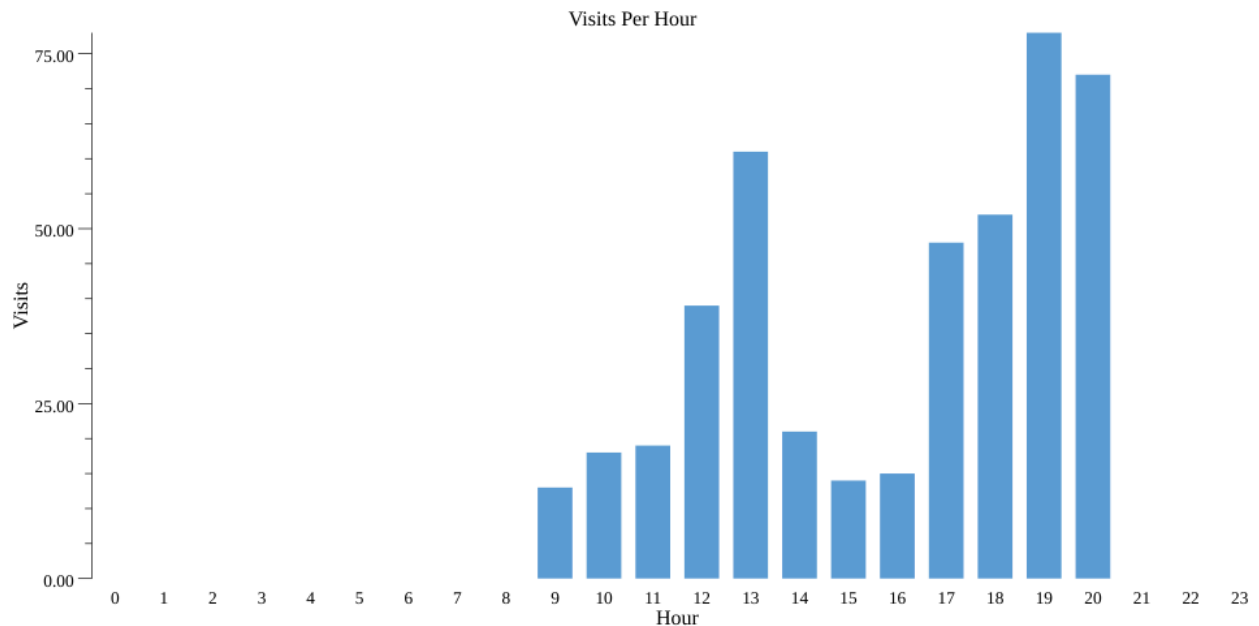
訪客 ID	進入時間	花費時間 (分鐘)	消費金額 (歐元)
1	上午 9:30:00	26	24
2	上午 11:54:00	53	17
3	下午 1:05:00	81	33

您會先使用下列程式碼範例中的 Beam，製作餐廳造訪時間的非私密長條圖。Scope 代表管道，我們對資料執行的每項新作業都會新增至 Scope。CountVisitsPerHour 會採用 Scope 和一系列造訪記錄，這些記錄在 Beam 中以 PCollection 表示。方法是在集合中套用 extractVisitHour 函式，擷取每次造訪的小時數。然後計算每個小時的發生次數並傳回。

```
func CountVisitsPerHour(s beam.Scope, col beam.PCollection) beam.PCollection {
    s = s.Scope("CountVisitsPerHour")
    visitHours := beam.ParDo(s, extractVisitHourFn, col)
    visitsPerHour := stats.Count(s, visitHours)
    return visitsPerHour
}

func extractVisitHourFn(v Visit) int {
    return v.TimeEntered.Hour()
}
```

這樣會在目前目錄中產生美觀的長條圖 (透過執行 `bazel run codelab -- --example="count" --input_file=$(pwd)/day_data.csv --output_stats_file=$(pwd)/count.csv --output_chart_file=$(pwd)/count.png`)，如下所示：`count.png`：



下一步是將管道和長條圖轉換為私人管道和長條圖。具體做法如下。

首先，在 `PCollection<V>` 上呼叫 `MakePrivateFromStruct`，取得 `PrivatePCollection<V>`。輸入的 `PCollection` 必須是結構體集合。我們需要輸入 `PrivacySpec` 和 `idFieldPath` 做為 `MakePrivateFromStruct` 的輸入內容。

```
spec := pbeam.NewPrivacySpec(epsilon, delta)
pCol := pbeam.MakePrivateFromStruct(s, col, spec, "VisitorID")
```

`PrivacySpec` 結構體會保留用於匿名化資料的差異化隱私參數 (`epsilon` 和 `delta`)。(您目前不必擔心這些問題，稍後會有選用章節，可供您進一步瞭解這些問題)。

`idFieldPath` 是結構體中的使用者 ID 欄位路徑 (在本例中為 `visit`)。在此，訪客的使用者 ID 是 `visit` 的 `VisitorID` 欄位。

接著，我們呼叫 `pbeam.Count()`，而非 `stats.Count()`。`pbeam.Count()` 會將 `CountParams` 結構體做為輸入內容，其中包含 `MaxValue` 等參數，這些參數會影響輸出內容的準確度。

```
visitsPerHour := pbeam.Count(s, visitHours, pbeam.CountParams{
    // Visitors can visit the restaurant once (one hour) a day
    MaxPartitionsContributed: 1,
    // Visitors can visit the restaurant once within an hour
    MaxValue: 1,
})
```

同樣地，`MaxPartitionsContributed` 會限制使用者可貢獻的不同造訪時數。我們預期他們每天最多只會造訪餐廳一次 (或不在意他們一天內造訪多次)，因此也將此值設為 1。我們會在選用章節中詳細說明這些參數。

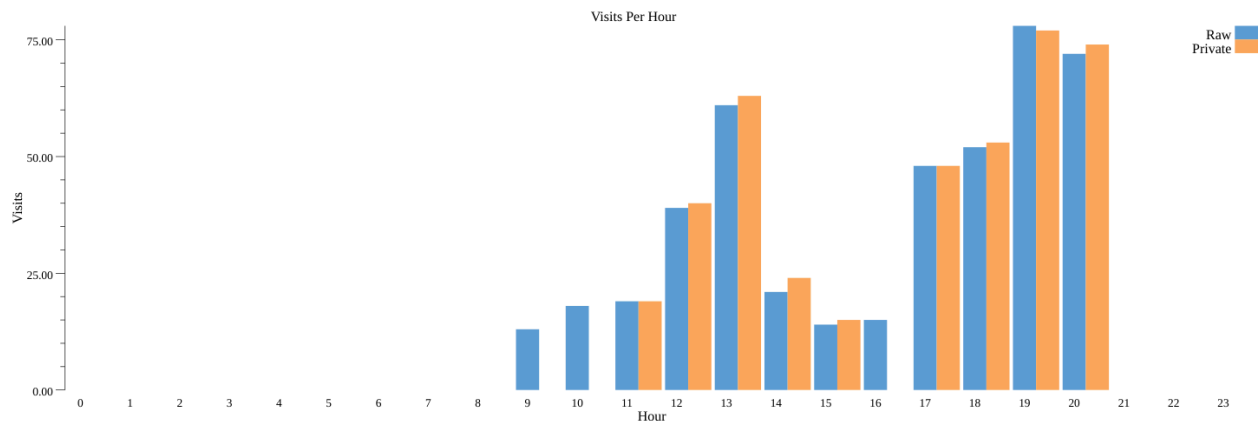
`MaxValue` 會限制單一使用者可為我們計算的值貢獻的次數。在本例中，我們要計算的是造訪時數，且我們預期使用者只會造訪餐廳一次 (或我們不在意使用者每小時造訪多次)，因此將這個參數設為 1。

最後的程式碼應如下所示：

```
func PrivateCountVisitsPerHour(s beam.Scope, col beam.PCollection) beam.PCollection {
    s = s.Scope("PrivateCountVisitsPerHour")
    // Create a Privacy Spec and convert col into a PrivatePCollection
    spec := pbeam.NewPrivacySpec(epsilon, delta)
    pCol := pbeam.MakePrivateFromStruct(s, col, spec, "VisitorID")

    visitHours := pbeam.ParDo(s, extractVisitHourFn, pCol)
    visitsPerHour := pbeam.Count(s, visitHours, pbeam.CountParams{
        // Visitors can visit the restaurant once (one hour) a day
        MaxPartitionsContributed: 1,
        // Visitors can visit the restaurant once within an hour
        MaxValue: 1,
    })
    return visitsPerHour
}
```

我們看到類似的長條圖 (count_dp.png)，代表差異化隱私統計資料 (先前的指令會執行非私有和私有管道)：



恭喜！您已計算出第一個差異隱私統計資料！

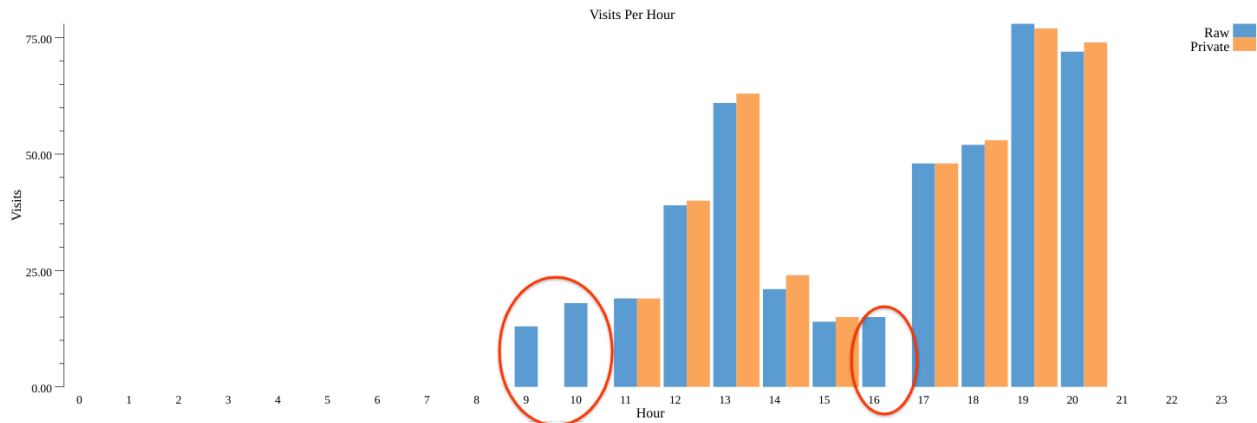
執行程式碼後取得的長條圖可能與這個不同。沒關係，由於差異化隱私權的雜訊，每次執行程式碼時，您都會取得不同的長條圖，但您會發現這些長條圖與我們原先的非私密長條圖大致相似。

請注意，為確保隱私權保障，請勿多次重新執行管道 (例如為了取得更好看的長條圖)。「計算多項統計資料」一節說明瞭不應重新執行管道的原因。

步驟 5 · 約 0 分鐘

5. 使用公開分區

在上一節中，您可能已注意到我們捨棄了某些分割區 (即時數) 的所有造訪次數 (資料)。



這是因為系統會進行分區選取/設定門檻，這是重要步驟，可確保輸出分區的存在與否取決於使用者資料本身時，系統能提供差異化隱私保證。在這種情況下，輸出內容中只要有區隔，就可能洩漏資料中存在個別使用者 (如需瞭解這為何會侵犯隱私權，請參閱這篇[網誌文章](#))。為避免這種情況，Privacy on Beam 只會保留使用者人數充足的分割區。

如果輸出分區清單不依附於私密使用者資料 (即為公開資訊)，我們就不需要這個分區選取步驟。以餐廳為例，我們知道餐廳的營業時間 (9:00 至 21:00)。

本範例的程式碼位於 `codelab/public_partitions.go` 中。

我們只會建立 9 到 21 之間的 PCollection (不含 9 和 21)，並將其輸入 `CountParams` 的 `PublicPartitions` 欄位：

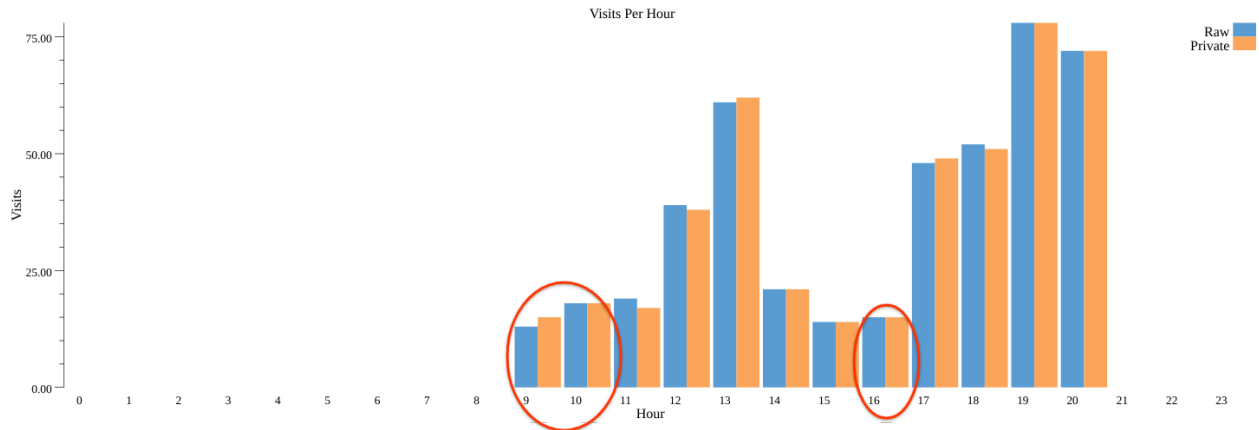
```
func PrivateCountVisitsPerHourWithPublicPartitions(s beam.Scope,
    col beam.PCollection) beam.PCollection {
    s = s.Scope("PrivateCountVisitsPerHourWithPublicPartitions")
    // Create a Privacy Spec and convert col into a PrivatePCollection
    spec := pbeam.NewPrivacySpec(epsilon, /* delta */ 0)
    pCol := pbeam.MakePrivateFromStruct(s, col, spec, "VisitorID")

    // Create a PCollection of output partitions, i.e. restaurant's work hours
    // (from 9 am till 9pm (exclusive)).
    hours := beam.CreateList(s, [12]int{9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
20})

    visitHours := pbeam.ParDo(s, extractVisitHourFn, pCol)
    visitsPerHour := pbeam.Count(s, visitHours, pbeam.CountParams{
        // Visitors can visit the restaurant once (one hour) a day
        MaxPartitionsContributed: 1,
        // Visitors can visit the restaurant once within an hour
        MaxValue: 1,
        // Visitors only visit during work hours
        PublicPartitions: hours,
    })
    return visitsPerHour
}
```

請注意，如果您使用公開分割區和 Laplace 雜訊 (預設)，可以將 `delta` 設為 0，如上所示。

```
使用公開分割區 (含 bazel run codelab -- --example="public_partitions" --  
input_file=$(pwd)/day_data.csv --output_stats_file=$(pwd)/public_partitions.csv --  
output_chart_file=$(pwd)/public_partitions.png) 執行管道時，我們會取得 (public_partitions_dp.png)：
```



如您所見，我們現在會保留先前捨棄的分割區 9、10 和 16，而不會保留公開分割區。

使用公開分區不僅能保留更多分區，與不使用公開分區相比，每個分區的雜訊也大約減少一半，因為分區選取時不會耗用任何隱私權預算，也就是 `epsilon` 和 `delta`。因此，與上次執行相比，原始計數和私人計數之間的差異略有減少。

使用公開分區時，請留意以下兩個重要事項：

1. 從原始資料衍生分割區清單時，請務必謹慎操作。如果沒有以差異化隱私的方式執行這項操作 (例如只是讀取使用者資料中的所有分割區清單)，管道就不再提供差異化隱私保證。如要瞭解如何以差異化隱私方式執行這項操作，請參閱下方的進階部分。
2. 如果部分公開分區沒有資料 (例如造訪次數)，系統會對這些分區套用雜訊，以維護差異化隱私。舉例來說，如果我們使用 0 到 24 小時 (而非 9 到 21 小時)，所有時數都會加入雜訊，即使沒有任何造訪，也可能會顯示一些造訪次數。

(進階) 從資料衍生分區

如果您在同一個管道中，使用相同的非公開輸出分割區清單執行多項匯總作業，可以先使用 `SelectPartitions()` 衍生分割區清單，然後將分割區做為 `PublicPartition` 輸入內容提供給各項匯總作業。從隱私權角度來看，這項做法不僅安全，還能減少雜訊，因為整個管道只會使用一次隱私公開程度上限來選取分區。

步驟 6 · 約 0 分鐘

6. 計算平均入住天數

我們現在知道如何以差異化隱私的方式計算項目，接下來要瞭解如何計算平均值。具體來說，我們現在要計算訪客的平均停留時間。

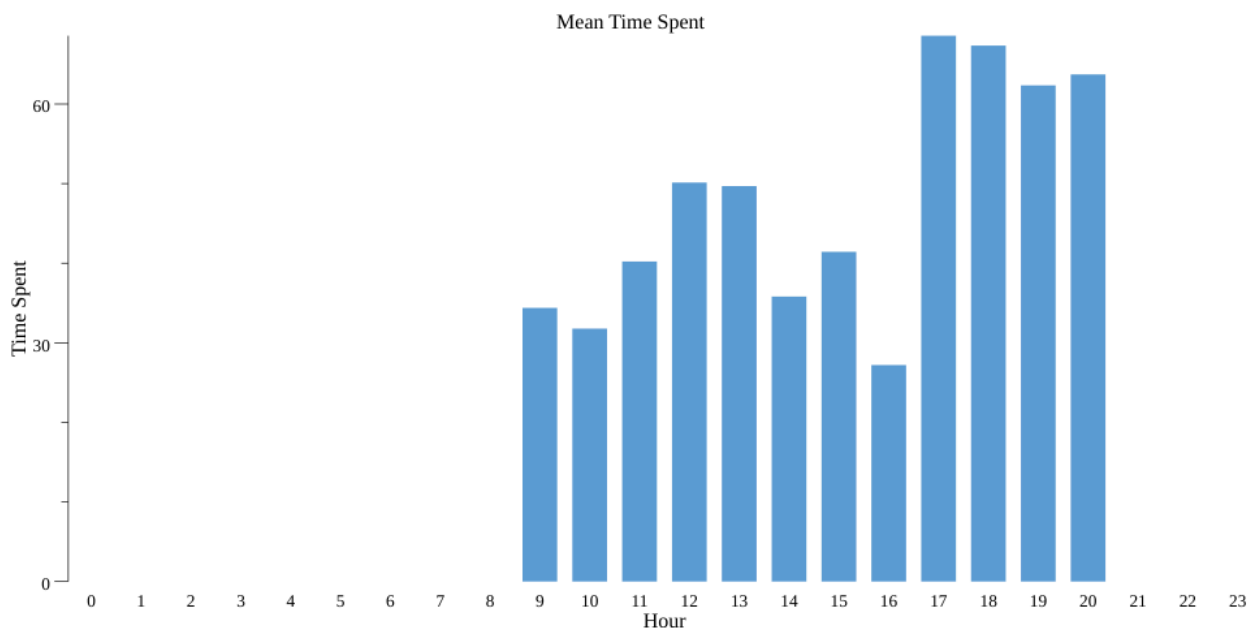
本範例的程式碼位於 `codelab/mean.go` 中。

一般來說，如要計算非私有停留時間平均值，我們會使用 `stats.MeanPerKey()`，並在預先處理步驟中將來訪的 `PCollection` 轉換為 `PCollection<K,V>`，其中 K 是來訪時間，V 則是訪客在餐廳停留的時間。

```
func MeanTimeSpent(s beam.Scope, col beam.PCollection) beam.PCollection {
    s = s.Scope("MeanTimeSpent")
    hourToTimeSpent := beam.ParDo(s, extractVisitHourAndTimeSpentFn, col)
    meanTimeSpent := stats.MeanPerKey(s, hourToTimeSpent)
    return meanTimeSpent
}

func extractVisitHourAndTimeSpentFn(v Visit) (int, int) {
    return v.TimeEntered.Hour(), v.MinutesSpent
}
```

這樣會在目前目錄中產生美觀的長條圖 (透過執行 `bazel run codelab -- --example="mean" --input_file=$(pwd)/day_data.csv --output_stats_file=$(pwd)/mean.csv --output_chart_file=$(pwd)/mean.png`)，如下所示：`mean.png`：



如要讓這項作業具有差異隱私權，我們再次將 `PCollection` 轉換為 `PrivatePCollection`，並將 `stats.MeanPerKey()` 替換為 `pbeam.MeanPerKey()`。與 `Count` 類似，我們有 `MeanParams`，可保留部分參數，例如 `MinValue` 和 `MaxValue`，這些參數會影響準確度。`MinValue` 和 `MaxValue` 代表我們為每位使用者對每個金鑰的貢獻設定的界限。

```

meanTimeSpent := pbeam.MeanPerKey(s, hourToTimeSpent, pbeam.MeanParams{
    // Visitors can visit the restaurant once (one hour) a day
    MaxPartitionsContributed: 1,
    // Visitors can visit the restaurant once within an hour
    MaxContributionsPerPartition: 1,
    // Minimum time spent per user (in mins)
    MinValue: 0,
    // Maximum time spent per user (in mins)
    MaxValue: 60,
})

```

在本例中，每個鍵代表一小時，值則是訪客花費的時間。我們將 `MinValue` 設為 0，因為我們預期訪客在餐廳停留的時間不會少於 0 分鐘。我們將 `MaxValue` 設為 60，也就是說，如果訪客停留時間超過 60 分鐘，我們會將該使用者停留時間視為 60 分鐘。

最後的程式碼應如下所示：

```

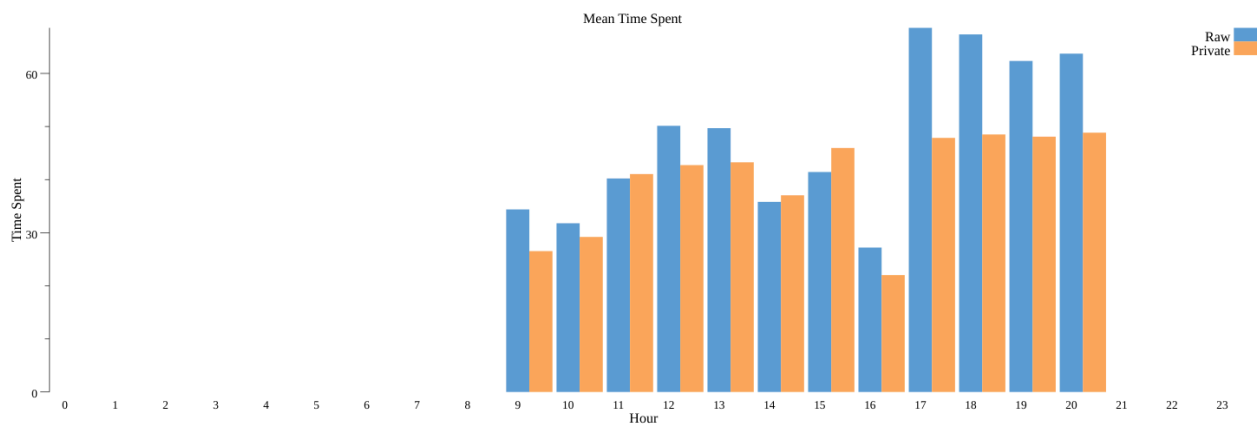
func PrivateMeanTimeSpent(s beam.Scope, col beam.PCollection) beam.PCollection {
    s = s.Scope("PrivateMeanTimeSpent")
    // Create a Privacy Spec and convert col into a PrivatePCollection
    spec := pbeam.NewPrivacySpec(epsilon, /* delta */ 0)
    pCol := pbeam.MakePrivateFromStruct(s, col, spec, "VisitorID")

    // Create a PCollection of output partitions, i.e. restaurant's work hours
    // (from 9 am till 9pm (exclusive)).
    hours := beam.CreateList(s, [12]int{9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
20})

    hourToTimeSpent := pbeam.ParDo(s, extractVisitHourAndTimeSpentFn, pCol)
    meanTimeSpent := pbeam.MeanPerKey(s, hourToTimeSpent, pbeam.MeanParams{
        // Visitors can visit the restaurant once (one hour) a day
        MaxPartitionsContributed: 1,
        // Visitors can visit the restaurant once within an hour
        MaxContributionsPerPartition: 1,
        // Minimum time spent per user (in mins)
        MinValue: 0,
        // Maximum time spent per user (in mins)
        MaxValue: 60,
        // Visitors only visit during work hours
        PublicPartitions: hours,
    })
    return meanTimeSpent
}

```

我們看到類似的長條圖 (`mean_dp.png`)，代表差異化隱私統計資料 (先前的指令會執行非私有和私有管道)：



同樣地，由於這是差異隱私作業，每次運作執行時都會得到不同的結果，與計數類似。但您可以發現，差異隱私權保護的停留時間與實際結果相差不遠。

步驟 7 · 約 0 分鐘

7. 計算每小時收益

我們還可以查看一天內每小時的收益，這也是一項有趣的統計資料。

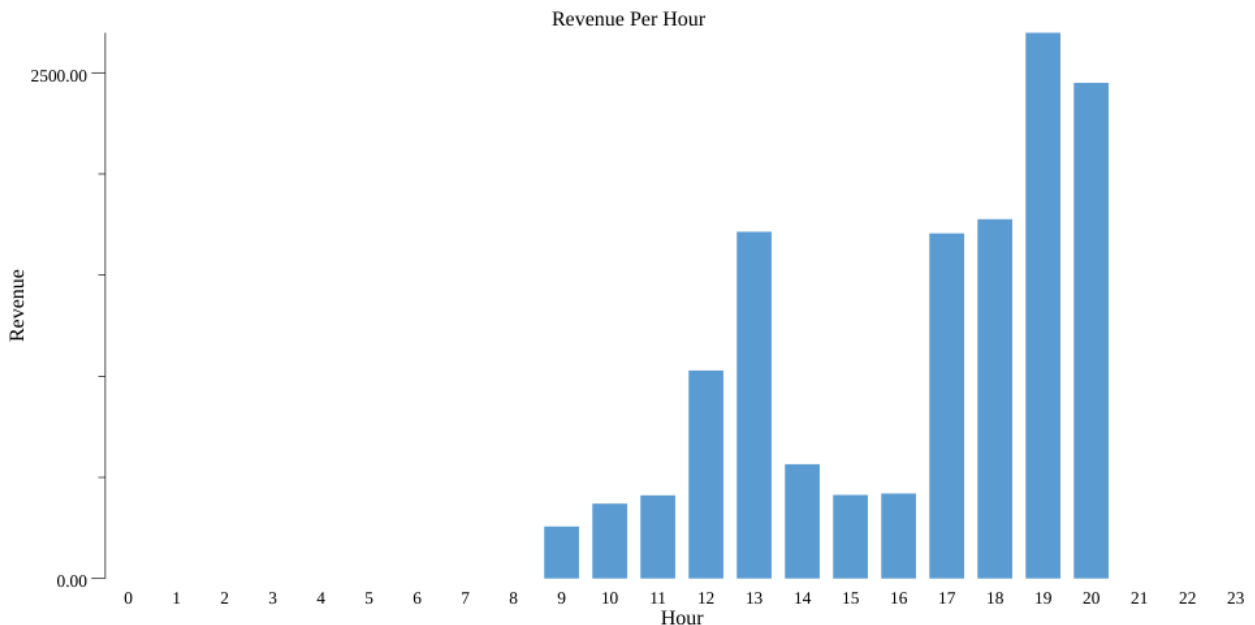
本範例的程式碼位於 `codelab/sum.go` 中。

同樣地，我們先從非私有版本開始。對模擬資料集進行一些前置處理後，我們就能建立 `PCollection<K,V>`，其中 `K` 是造訪時間，`V` 則是訪客在餐廳消費的金額：如要計算每小時的非私密收益，只要呼叫 `stats.SumPerKey()` 即可加總訪客消費的所有金額：

```
func RevenuePerHour(s beam.Scope, col beam.PCollection) beam.PCollection {
    s = s.Scope("RevenuePerHour")
    hourToMoneySpent := beam.ParDo(s, extractVisitHourAndMoneySpentFn, col)
    revenues := stats.SumPerKey(s, hourToMoneySpent)
    return revenues
}

func extractVisitHourAndMoneySpentFn(v Visit) (int, int) {
    return v.TimeEntered.Hour(), v.MoneySpent
}
```

這樣會在目前目錄中產生美觀的長條圖 (透過執行 `bazel run codelab -- --example="sum" --input_file=$(pwd)/day_data.csv --output_stats_file=$(pwd)/sum.csv --output_chart_file=$(pwd)/sum.png`)，如下所示：`sum.png`：



如要讓這項作業具有差異隱私權，我們再次將 `PCollection` 轉換為 `PrivatePCollection`，並將 `stats.SumPerKey()` 替換為 `pbeam.SumPerKey()`。與 `Count` 和 `MeanPerKey` 類似，我們有 `SumParams`，其中包含 `MinValue` 和 `MaxValue` 等參數，這些參數會影響準確度。


```

revenues := pbeam.SumPerKey(s, hourToMoneySpent, pbeam.SumParams{
    // Visitors can visit the restaurant once (one hour) a day
    MaxPartitionsContributed: 1,
    // Minimum money spent per user (in euros)
    MinValue:                  0,
    // Maximum money spent per user (in euros)
    MaxValue:                  40,
})

```

在本例中，`MinValue` 和 `MaxValue` 代表每位訪客支出的金額範圍。我們將 `MinValue` 設為 0，因為我們預期訪客在餐廳的消費金額不會低於 0 歐元。我們將 `MaxValue` 設為 40，也就是說，如果訪客消費超過 40 歐元，我們會將該使用者視為消費 40 歐元。

最後的程式碼應如下所示：

```

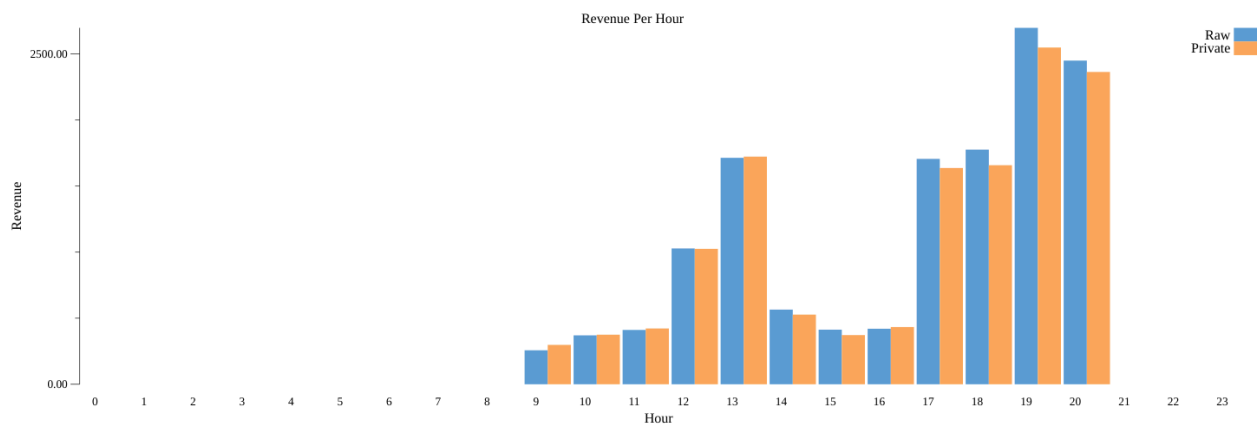
func PrivateRevenuePerHour(s beam.Scope, col beam.PCollection) beam.PCollection {
    s = s.Scope("PrivateRevenuePerHour")
    // Create a Privacy Spec and convert col into a PrivatePCollection
    spec := pbeam.NewPrivacySpec(epsilon, /* delta */ 0)
    pCol := pbeam.MakePrivateFromStruct(s, col, spec, "VisitorID")

    // Create a PCollection of output partitions, i.e. restaurant's work hours
    // (from 9 am till 9pm (exclusive)).
    hours := beam.CreateList(s, [12]int{9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
20})

    hourToMoneySpent := pbeam.ParDo(s, extractVisitHourAndMoneySpentFn, pCol)
    revenues := pbeam.SumPerKey(s, hourToMoneySpent, pbeam.SumParams{
        // Visitors can visit the restaurant once (one hour) a day
        MaxPartitionsContributed: 1,
        // Minimum money spent per user (in euros)
        MinValue:                  0,
        // Maximum money spent per user (in euros)
        MaxValue:                  40,
        // Visitors only visit during work hours
        PublicPartitions:          hours,
    })
    return revenues
}

```

我們看到類似的長條圖 (`sum_dp.png`)，代表差異化隱私統計資料 (先前的指令會執行非私有和私有管道)：



同樣地，由於這是差異隱私作業，每次執行時都會得到不同的結果，這與計數和平均值類似。但您可以發現，差異隱私結果與每小時實際收益非常接近。

8. 計算多項統計資料

在多數情況下，您可能會想對相同的基礎資料計算多項統計資料，就像您對計數、平均值和總和所做的計算一樣。通常在單一 Beam 管道和單一二進位檔中執行這項作業，會更簡潔且容易。你也可以使用 Beam 的隱私權功能執行這項操作。您可以編寫單一管道來執行轉換和運算，並為整個管道使用單一 `PrivacySpec`。

這樣不僅更方便，也更能保護隱私。`PrivacySpec` 如果您還記得我們提供給 `PrivacySpec` 的 `epsilon` 和 `delta` 參數，這些參數代表所謂的「隱私公開程度上限」，也就是您洩漏基礎資料中使用者隱私權的程度。

請務必記得，隱私權預算是累加的：如果您使用特定 `epsilon` ϵ 和 `delta` δ 執行管道一次，就會用掉 (ϵ, δ) 預算。如果再次執行，總預算支出為 $(2\epsilon, 2\delta)$ 。同樣地，如果您使用 (ϵ, δ) 的 `PrivacySpec` (並連續使用隱私權預算) 計算多項統計資料，則總預算為 $(2\epsilon, 2\delta)$ 。這表示您正在降低隱私權保障。

為規避這項限制，如要根據相同的基礎資料計算多項統計資料，請使用單一 `PrivacySpec`，並設定您想使用的總預算。接著，您需要為每個匯總指定要使用的 `epsilon` 和 `delta`。最終，您會獲得相同的整體隱私權保障；但特定匯總的 `Epsilon` 和 `Delta` 越高，準確度就越高。

如要查看這項操作，我們可以在單一管道中，計算先前分別計算的三項統計資料 (計數、平均值和總和)。

本範例的程式碼位於 `code-lab/multiple.go`。請注意，我們在三種匯總作業之間平均分配總 (ϵ, δ) 預算：

```

func ComputeCountMeanSum(s beam.Scope, col beam.PCollection) (visitsPerHour,
meanTimeSpent, revenues beam.PCollection) {
    s = s.Scope("ComputeCountMeanSum")
    // Create a Privacy Spec and convert col into a PrivatePCollection
    // Budget is shared by count, mean and sum.
    spec := pbeam.NewPrivacySpec(epsilon, /* delta */ 0)
    pCol := pbeam.MakePrivateFromStruct(s, col, spec, "VisitorID")

    // Create a PCollection of output partitions, i.e. restaurant's work hours
    // (from 9 am till 9pm (exclusive)).
    hours := beam.CreateList(s, [12]int{9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
20})

    visitHours := pbeam.ParDo(s, extractVisitHourFn, pCol)
    visitsPerHour = pbeam.Count(s, visitHours, pbeam.CountParams{
        Epsilon:          epsilon / 3,
        Delta:             0,
        // Visitors can visit the restaurant once (one hour) a day
        MaxPartitionsContributed: 1,
        // Visitors can visit the restaurant once within an hour
        MaxValue:          1,
        // Visitors only visit during work hours
        PublicPartitions:   hours,
    })

    hourToTimeSpent := pbeam.ParDo(s, extractVisitHourAndTimeSpentFn, pCol)
    meanTimeSpent = pbeam.MeanPerKey(s, hourToTimeSpent, pbeam.MeanParams{
        Epsilon:          epsilon / 3,
        Delta:             0,
        // Visitors can visit the restaurant once (one hour) a day
        MaxPartitionsContributed: 1,
        // Visitors can visit the restaurant once within an hour
        MaxContributionsPerPartition: 1,
        // Minimum time spent per user (in mins)
        MinValue:          0,
        // Maximum time spent per user (in mins)
        MaxValue:          60,
        // Visitors only visit during work hours
        PublicPartitions:   hours,
    })

    hourToMoneySpent := pbeam.ParDo(s, extractVisitHourAndMoneySpentFn, pCol)
    revenues = pbeam.SumPerKey(s, hourToMoneySpent, pbeam.SumParams{
        Epsilon:          epsilon / 3,
        Delta:             0,
        // Visitors can visit the restaurant once (one hour) a day
        MaxPartitionsContributed: 1,
        // Minimum money spent per user (in euros)

```

```
    MinValue:          0,  
    // Maximum money spent per user (in euros)  
    MaxValue:          40,  
    // Visitors only visit during work hours  
    PublicPartitions:  hours,  
  })  
  
  return visitsPerHour, meanTimeSpent, revenues  
}
```

9. (選用) 調整差異隱私權參數

您在本程式碼研究室中看到許多參數，例如 `epsilon`、`delta`、`maxPartitionsContributed` 等。我們大致可將這些參數分為兩類：隱私權參數和實用參數。

隱私權參數

`Epsilon` 和 `delta` 是用來量化差異化隱私所提供隱私權的參數。更精確地說，`epsilon` 和 `delta` 是用來衡量潛在攻擊者查看匿名輸出內容時，可獲得多少基礎資料資訊。`epsilon` 和 `delta` 越高，攻擊者就越能瞭解基礎資料，這會造成隱私權風險。

另一方面，`epsilon` 和 `delta` 值越低，您就越需要在輸出內容中加入雜訊，才能確保匿名性。此外，每個分割區中也需要有更多不重複使用者，才能將該分割區保留在匿名輸出內容中。因此，實用性和隱私權之間存在取捨關係。

在 Beam 的隱私權設定中，您需要在 `PrivacySpec` 中指定隱私權總預算，並確保匿名輸出內容符合隱私權保障。但請注意，如要確保隱私權保障措施有效，請按照本程式碼研究室的建議，為每個匯總作業分別設定 `PrivacySpec`，或多次執行管道，以免預算用盡。

如要進一步瞭解差異隱私權和隱私權參數的意義，請參閱[文獻](#)。

公用程式參數

這些參數不會影響隱私權保障 (只要正確遵循有關如何在 Beam 上使用 Privacy 的建議)，但會影響準確度，進而影響輸出內容的實用性。這些參數會提供在每個匯總的 `Params` struct 中，例如 `CountParams`、`SumParams` 等。這些參數用於調整加入的雜訊。

`Params` 中提供的公用程式參數 `MaxPartitionsContributed` 適用於所有匯總。分區對應於 Privacy On Beam 彙整作業輸出的 PCollection 索引鍵，即 `Count`、`SumPerKey` 等。因此，`MaxPartitionsContributed` 會限制使用者可在輸出內容中提供的不同索引鍵值數量。如果使用者對基礎資料中的鍵貢獻超過 `MaxPartitionsContributed` 個，系統會捨棄部分貢獻，確保使用者貢獻的鍵數量正好是 `MaxPartitionsContributed` 個。

與 `MaxPartitionsContributed` 類似，大多數的匯總都有 `MaxContributionsPerPartition` 參數。這些值會以 `Params` 結構體的形式提供，且每個匯總都可能具有不同的值。與 `MaxPartitionsContributed` 不同，`MaxContributionsPerPartition` 會限制每位使用者對每個鍵的貢獻。也就是說，每位使用者只能為每個鍵提供 `MaxContributionsPerPartition` 值。

加到輸出的雜訊會依 `MaxPartitionsContributed` 和 `MaxContributionsPerPartition` 調整比例，因此這裡需要取捨：`MaxPartitionsContributed` 和 `MaxContributionsPerPartition` 越大，保留的資料就越多，但結果的雜訊也會越多。

部分匯總作業需要 `MinValue` 和 `MaxValue`。這些限制會指定每位使用者所占資料量的範圍。如果使用者提供的價值低於 `MinValue`，系統會將該價值調高至 `MinValue`。同樣地，如果使用者提供的數值大於 `MaxValue`，該值會被限制在 `MaxValue`。也就是說，如要保留更多原始值，就必須指定較大的界限。與 `MaxPartitionsContributed` 和 `MaxContributionsPerPartition` 類似，雜訊會根據邊界大小進行縮放，因此邊界越大，保留的資料就越多，但結果雜訊也會越多。

我們要討論的最後一個參數是 `NoiseKind`。Privacy On Beam 支援兩種不同的雜訊機制：`GaussianNoise` 和 `LaplaceNoise`。兩者各有優缺點，但 Laplace 分布在貢獻範圍較低時，實用性較高，因此 Privacy On Beam 預設使用 Laplace 分布。不過，如要使用高斯分布雜訊，可以提供 `Params` 變數。 `pbeam.GaussianNoise{}`

10. 摘要

做得好！您已完成 Privacy on Beam 程式碼研究室。您學到了許多關於差異化隱私和 Beam 隱私權的知識：

- 透過呼叫 `MakePrivateFromStruct`，將 `PCollection` 轉換為 `PrivatePCollection`。
- 使用 `Count` 計算差異隱私計數。
- 使用 `MeanPerKey` 計算差異隱私平均值。
- 使用 `SumPerKey` 計算差異隱私總和。
- 在單一管道中，使用單一 `PrivacySpec` 計算多項統計資料。
- (選用) 自訂 `PrivacySpec` 和匯總參數 (`CountParams`, `MeanParams`, `SumParams`)。

不過，您還可以使用 Beam 上的 Privacy 執行更多匯總作業 (例如分位數、計算不重複值)！如要進一步瞭解這些函式庫，請前往 [GitHub 存放區](#) 或 [godoc](#)。

如有時間，請填寫[問卷調查](#)，提供對程式碼研究室的意見。
